Résume

Ce projet, intitulé « Under the Hood : Conception et Déploiement d'un Web Application Firewall », a été réalisé dans le cadre du filière de cybersécurité de l'ENSAM CASABLANCA.

L'objectif principal était de s'éloigner des interfaces graphiques commerciales pour explorer les mécanismes fondamentaux d'interception et d'analyse du trafic HTTP.

Le travail s'articule autour de quatre phases clés définies par le cahier des charges :

Audit de vulnérabilité : Déploiement d'une application cible (DVWA) et validation par des tests d'intrusion manuels (SQL Injection et Cross-Site Scripting) pour prouver l'état initial d'insécurité.
Implémentation Open-Source : Mise en œuvre de ModSecurity avec l'OWASP Core Rule Set (CRS) en tant que reverse proxy, permettant d'observer une défense industrielle standardisée. Ingénierie d'une solution personnalisée : Développement en Python d'un WAF "maison" agissant comme reverse proxy.
Ce script intègre un moteur d'inspection complet (URI, Headers, Body) et une logique de détection basée sur des expressions régulières pour bloquer les menaces avec une réponse HTTP 403 personnalisée.
Analyse et "Showdown" : Évaluation comparative des performances à l'aide de scanners automatisés pour mesurer l'efficacité de blocage et les taux de faux positifs entre l'application non protégée, le WAF open-source et la solution personnalisée.
Le rapport conclut par une analyse critique comparant FortiWeb aux solutions étudiées, en évaluant les compromis entre facilité de déploiement, fonctionnalités avancées (Machine Learning vs Regex) et maintenance.

Abstract

This project, entitled "Under the Hood: Design and Deployment of a Web Application Firewall", was carried out as part of the cybersecurity Specialization at ENSAM CASABLANCA.

The main objective was to move away from commercial graphical interfaces in order to explore the fundamental mechanisms of HTTP traffic interception and analysis.

The work is organized around four key phases defined in the project specifications:

Vulnerability Audit: Deployment of a target application (DVWA) and validation through manual penetration tests (SQL Injection and Cross-Site Scripting) to demonstrate the initial insecure state.

Open-Source Implementation: Implementation of ModSecurity with the OWASP Core Rule Set (CRS) as a reverse proxy. This allowed us to observe a standardized industrial-grade defense.

Custom Solution Engineering: Development in Python of a custom ("home-made") WAF acting as a reverse proxy. This script integrates a complete inspection engine (URI, Headers, Body) and a detection logic based on regular expressions to block threats, returning a custom HTTP 403 response.

Analysis and "Showdown": Comparative performance evaluation using automated scanners to measure blocking effectiveness and false positive rates between the unprotected application, the open-source WAF, and the custom solution.

The report concludes with a critical analysis comparing FortiWeb to the solutions studied, evaluating the trade-offs between ease of deployment, advanced features (Machine Learning vs Regex), and maintenance.

SOMMAIRE :

REMERCIEMENTS 2

RESUME 3

ABSTRACT 4

SOMMAIRE 5

INTRODUCTION GÉNÉRALE 6

CHAPITRE 1 : ÉTUDE DE L'ART ET ANALYSE DES BESOINS 7

CHAPITRE 2 : ARCHITECTURE ET CONFIGURATION DE L'ENVIRONNEMENT (SETUP) 12

CHAPITRE 3 : CONCEPTION ET DÉVELOPPEMENT DU WAF PERSONNALISÉ 18

CHAPITRE 4 : PREUVE DE CONCEPT (POC) ET VALIDATION 24

CHAPITRE 5 : L'ULTIME COMPARAISON (ANALYSE ANALYTIQUE) 31

CONCLUSION GENERALE 35

CODES 36

BIBLIOGRAPHIE ET WEBOGRAPHIE 39

INTRODUCTION GÉNÉRALE

Dans un paysage numérique où les menaces applicatives évoluent plus vite que les infrastructures de défense, la maîtrise des mécanismes de filtrage du trafic Web est devenue une compétence vitale pour l'ingénieur en cybersécurité.

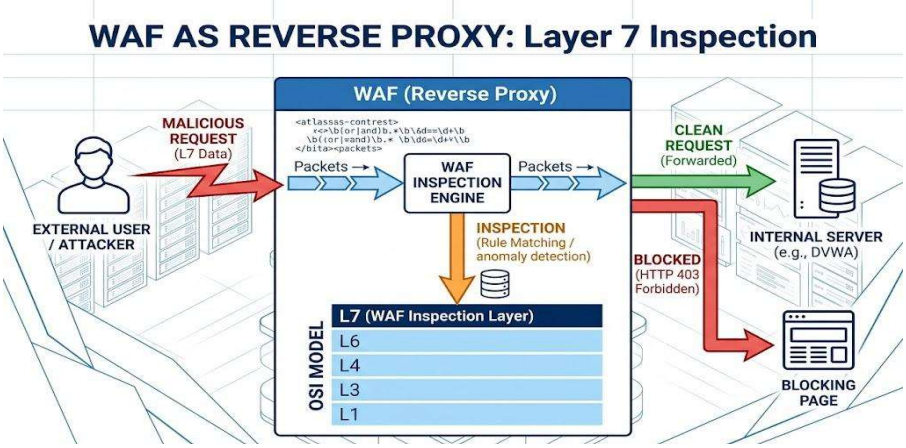
Ce projet, intitulé « Under the Hood », s'écarte des solutions commerciales "clés en main" comme FortiWeb pour explorer la réalité technique des Web Application Firewalls (WAF). L'objectif est de sécuriser une application délibérément vulnérable, DVWA (Damn Vulnerable Web App), déployée dans un environnement conteneurisé Docker.

La problématique centrale consiste à concevoir une architecture capable d'intercepter des requêtes malveillantes, telles que des injections SQL ou des attaques XSS, sans perturber le trafic légitime des utilisateurs.

Pour répondre à ce défi, nous avons mis en place une double ligne de défense. D'une part, l'implémentation d'une solution industrielle open-source basée sur ModSecurity et le Core Rule Set (CRS) de l'OWASP, configurée en mode Reverse Proxy. D'autre part, le développement complet d'un WAF personnalisé en Python, ce script « fait maison » agit comme une sentinelle réseau sur le port 5000, inspectant minutieusement les URI, les en-têtes et le corps des requêtes POST via des expressions régulières (Regex) pour bloquer les menaces avec une réponse HTTP 403 personnalisée.

Enfin, ce rapport présente une phase de confrontation technique (« The Showdown ») où l'efficacité de nos défenses est mesurée par des scanners de vulnérabilités automatisés, cette démarche nous permet de dresser une analyse comparative rigoureuse entre la puissance du Machine Learning des solutions propriétaires, la robustesse des règles communautaires open-source et la flexibilité d'un moteur de détection personnalisé, afin d'évaluer les compromis réels entre performance, coût et maintenance.

CHAPITRE I :
ÉTUDE DE L'ART ET ANALYSE DES BESOINS



Introduction

Ce chapitre vise à établir le cadre théorique de la sécurité des applications Web.

Il définit les menaces ciblées par ce projet et explique le rôle fondamental du Web Application Firewall (WAF) dans la défense en profondeur .

1. Panorama des menaces Web:

La surface d'attaque des applications Web modernes est vaste. Conformément au cahier des charges, nous nous concentrons sur les vulnérabilités majeures identifiées par l'OWASP.

1.1. L'Injection SQL (SQLi)

Définition : Cette attaque consiste à insérer ou « injecter » des requêtes SQL malveillantes via les champs d'entrée de l'application. ***Mécanisme*** : L'attaquant tente de manipuler la requête SQL originale pour contourner l'authentification ou extraire des données sensibles. ***Exemple concret*** : L'usage d'une tautologie comme ' OR '1'='1 dans un formulaire de connexion pour forcer l'accès sans mot de passe.

1.2. Le Cross-Site Scripting (XSS)

Définition : Le XSS permet à un attaquant d'injecter des scripts malveillants (souvent du JavaScript) dans des pages Web consultées par d'autres utilisateurs.
Impact : Vol de cookies de session, redirection vers des sites malveillants ou défaçage de page.
Exemple concret : L'insertion de la balise <script>alert('XSS')</script> dans un livre d'or ou un profil utilisateur.

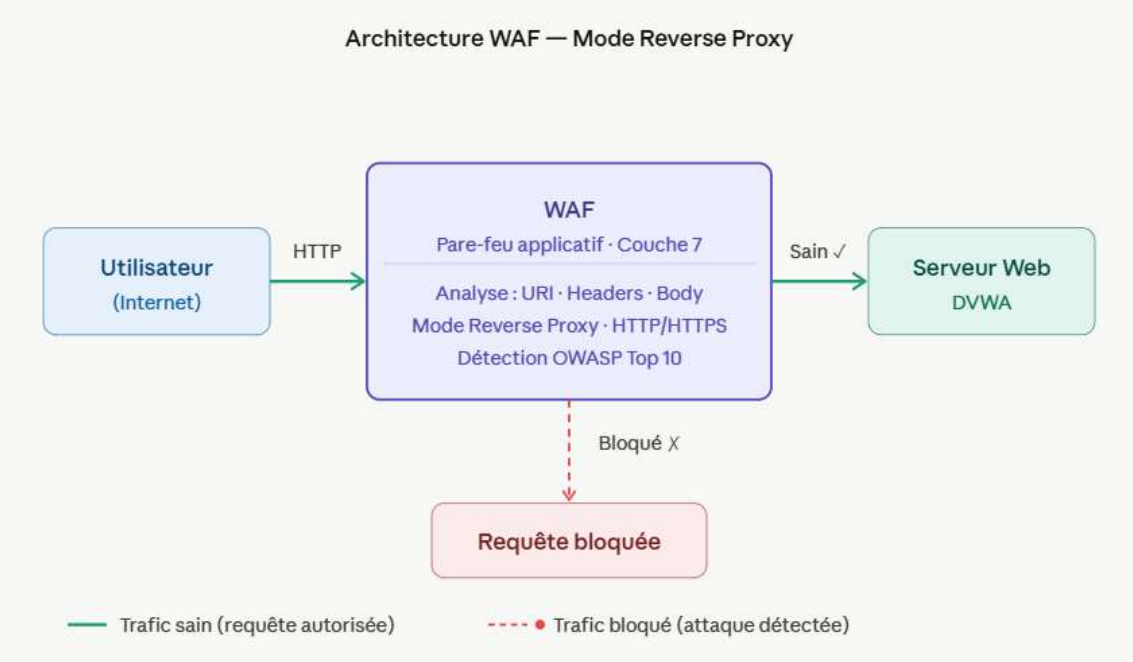
2. Théorie des Web Application Firewalls (WAF):

Pour contrer ces menaces, le WAF agit comme un filtre spécialisé entre l'utilisateur et le serveur d'application.

2.1. Le concept de Reverse Proxy

Fonctionnement : Contrairement à un proxy classique, le reverse proxy est placé devant les serveurs Web. Il reçoit toutes les requêtes entrantes pour le compte du serveur cible.

Rôle sécuritaire : Il masque l'adresse IP réelle du serveur interne et permet d'inspecter le trafic avant qu'il n'atteigne l'application.

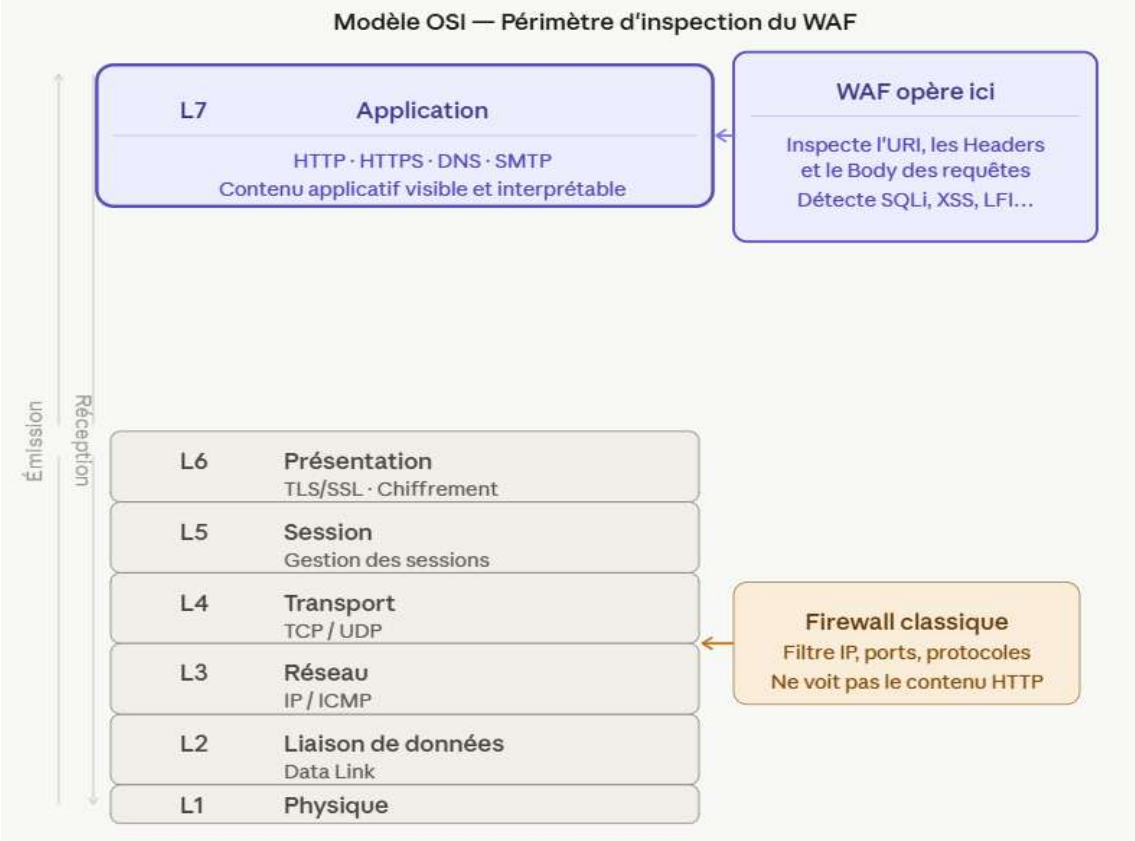


2.2. Inspection en couche 7 (Modèle OSI)

Différence avec un Firewall classique : Alors qu'un pare-feu traditionnel filtre les ports et les adresses IP (Couches 3 et 4), le WAF analyse le contenu des protocoles applicatifs (Couche 7).

Éléments inspectés :

L'URI : Détection de tentatives de traversée de répertoire (Path Traversal).
Les Headers : Vérification de la légitimité des en-têtes HTTP (User-Agent, Cookies).
Le Body : Analyse des données envoyées via les formulaires (POST data) pour y déceler des signatures malveillantes.



3. Analyse des besoins et critères de succès:

Cette section définit les exigences fonctionnelles pour les solutions déployées dans les phases suivantes :

3.1. Identification des actifs à protéger:

L'actif principal est l'application **DVWA** (Damn Vulnerable Web App), qui contient des données utilisateurs et des fonctionnalités sensibles.

3.2. Définition des critères de succès:

Pour que le projet soit considéré comme réussi, les solutions (ModSecurity et WAF Custom) doivent répondre aux critères suivants :

Efficacité de détection : Bloquer au minimum les attaques SQLi et XSS manuelles réalisées en phase de test.

Disponibilité : Ne pas interrompre le trafic légitime (minimiser les faux positifs).

Journalisation : Générer un log précis pour chaque blocage incluant l'IP, la date et le payload détecté.

Conclusion

En somme, ce premier chapitre a permis de définir le périmètre de sécurité en identifiant les vulnérabilités critiques que sont l'injection SQL et le Cross-Site Scripting, tout en justifiant l'utilisation d'une architecture de Reverse Proxy pour assurer une inspection granulaire en couche 7. L'analyse des besoins souligne que la protection de l'application DVWA ne peut reposer sur un pare-feu traditionnel, mais nécessite un dispositif capable de décoder et de filtrer le contenu applicatif en temps réel. Dès lors, les bases théoriques étant posées, nous pouvons passer à la phase de mise en œuvre technique de l'infrastructure Docker, afin de déployer les solutions de défense open-source et personnalisées qui permettront de répondre à ces exigences de sécurité.

CHAPITRE 2 :
ARCHITECTURE ET CONFIGURATION DE L'ENVIRONNEMENT (SETUP)

Introduction

Ce chapitre détaille la construction de l'infrastructure de sécurité. Il décrit la topologie réseau, le processus de conteneurisation et la configuration initiale de la cible et des systèmes de défense.

1. Infrastructure Système et Topologie Réseau:

Pour garantir une isolation parfaite et une flexibilité de déploiement, l'intégralité de l'environnement a été construite sur une machine hôte Kali Linux en utilisant la technologie de conteneurisation Docker.

1.1. Choix de la Conteneurisation :

L'utilisation de Docker permet de simuler une architecture réseau complexe (micro-services) sur une seule machine physique tout en assurant que les dépendances du WAF n'interfèrent pas avec celles de l'application cible.

1.2. Topologie Réseau (Mode Host):

Afin de lever les barrières d'isolation réseau classiques de Docker et permettre une communication fluide entre les différents composants, le mode réseau --net=host a été privilégié.

- **Flux entrant :** L'utilisateur (ou l'attaquant) accède aux services via l'adresse IP de la machine hôte.
- **Adressage des ports :**
  - **Port 80 :** Serveur cible (DVWA).
  - **Port 5000 :** WAF Personnalisé (Python).
  - **Port 8081 :** WAF Open-Source (ModSecurity).

2. Déploiement de l'Application Cible (Phase 1) :

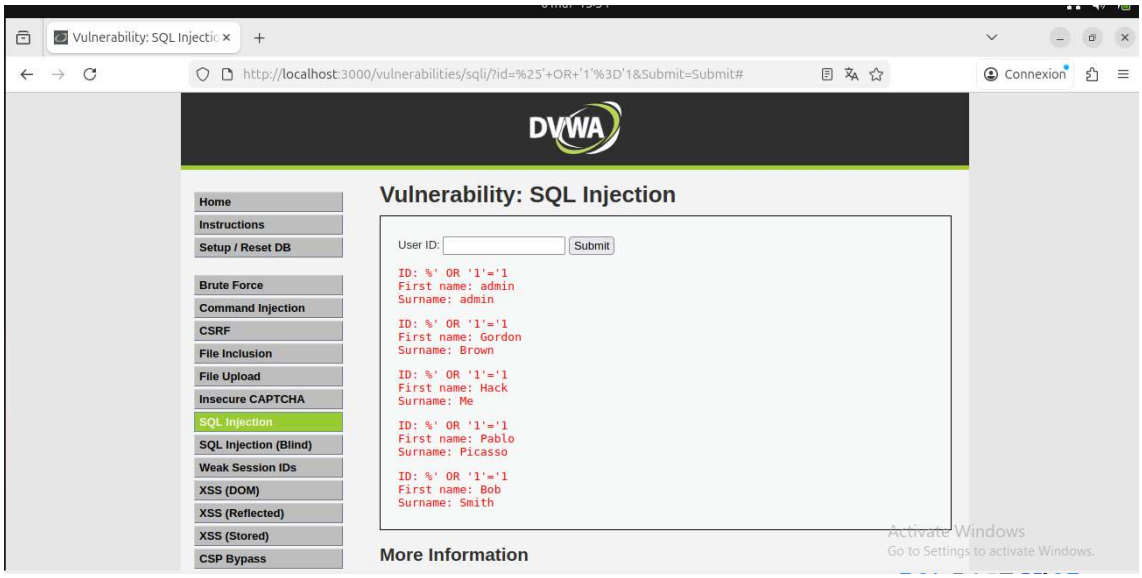
L'application **Damn Vulnerable Web App (DVWA)** a été déployée via l'image conteneurisée vulnerables/web-dvwa.

- **Configuration :** Initialisation de la base de données MySQL interne et paramétrage du niveau de sécurité sur "Low" pour valider les exploits initiaux.

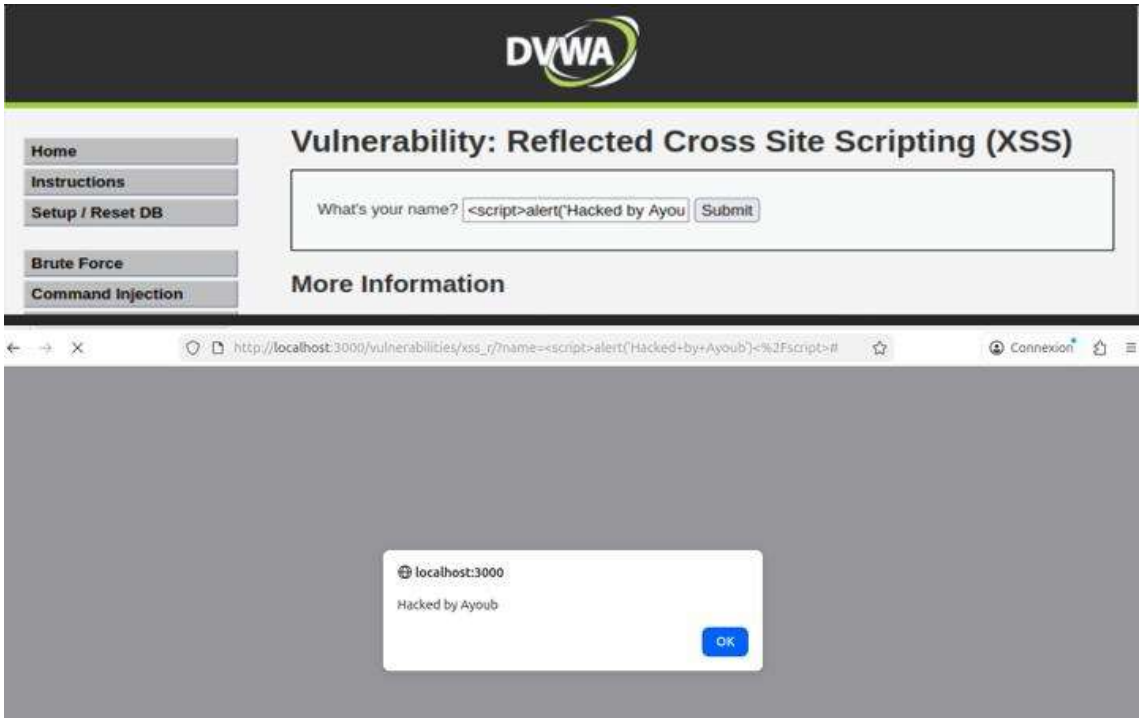
2.2. Validation de l'état vulnérable (Preuve de Concept)

Avant toute sécurisation, des tests d'intrusion manuels ont été effectués pour prouver que l'application est exposée:

- **SQLi :** Injection réussie dans le formulaire de recherche d'utilisateurs.



- **XSS :** Exécution d'un script alert() via le livre d'or (Stored XSS)



3. Mise en œuvre du WAF Open-Source (Phase 2):

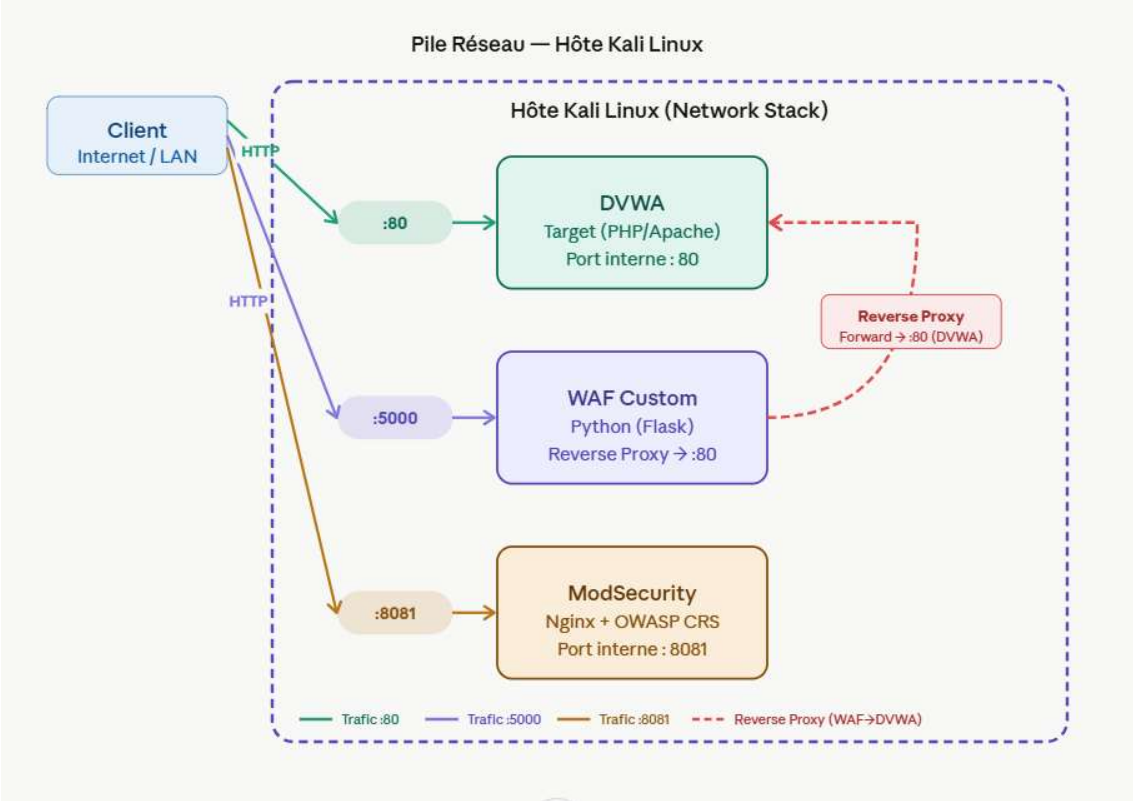
3.1. Déploiement de ModSecurity

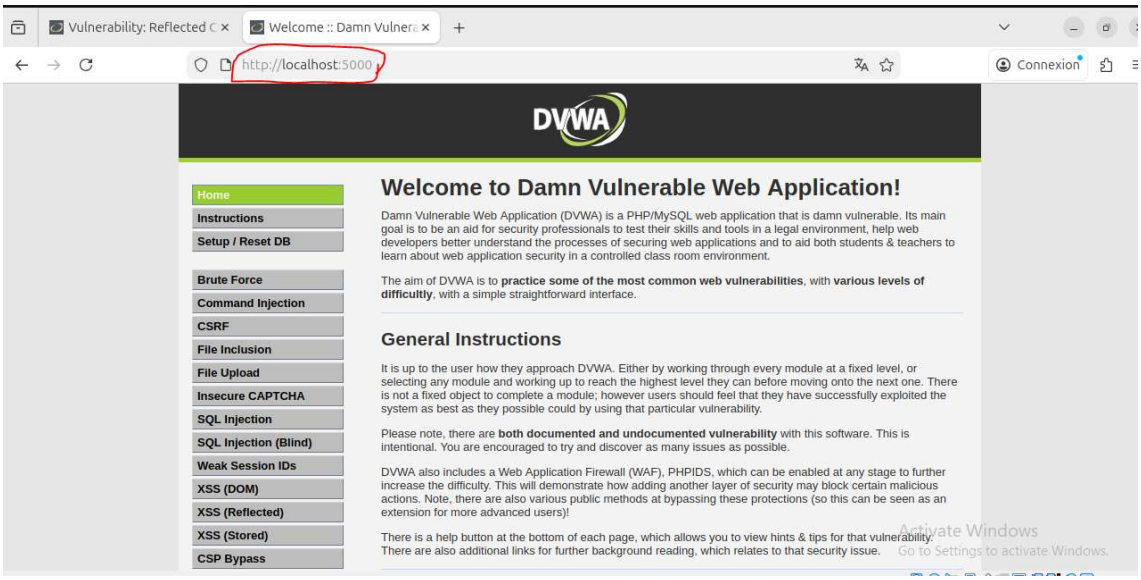
La solution open-source retenue est **ModSecurity** embarqué dans un serveur **Nginx**, déployé via l'image owasp/modsecurity-crs:nginx.

3.2. Implémentation de l'OWASP Core Rule Set (CRS)

Le WAF a été configuré pour utiliser les règles **CRS (v3/v4)**, qui fournissent une protection générique contre les attaques du Top 10 OWASP sans nécessiter de signatures spécifiques à l'application.

- **Configuration du Reverse Proxy :** Le paramètre PROXY_LOCATION a été défini sur http://127.0.0.1:80 pour rediriger le trafic sain vers DVWA après inspection.





Reverse Proxy fonctionne

Conclusion

L'architecture ainsi mise en place respecte scrupuleusement les principes de la défense en profondeur. Avec une cible vulnérable fonctionnelle et un premier bouclier industriel opérationnel, l'environnement est prêt pour la phase de développement du WAF personnalisé, qui viendra compléter ce dispositif en offrant une granularité de contrôle totale sur les flux HTTP.

CHAPITRE 3 :
CONCEPTION ET DÉVELOPPEMENT DU WAF PERSONNALISÉ

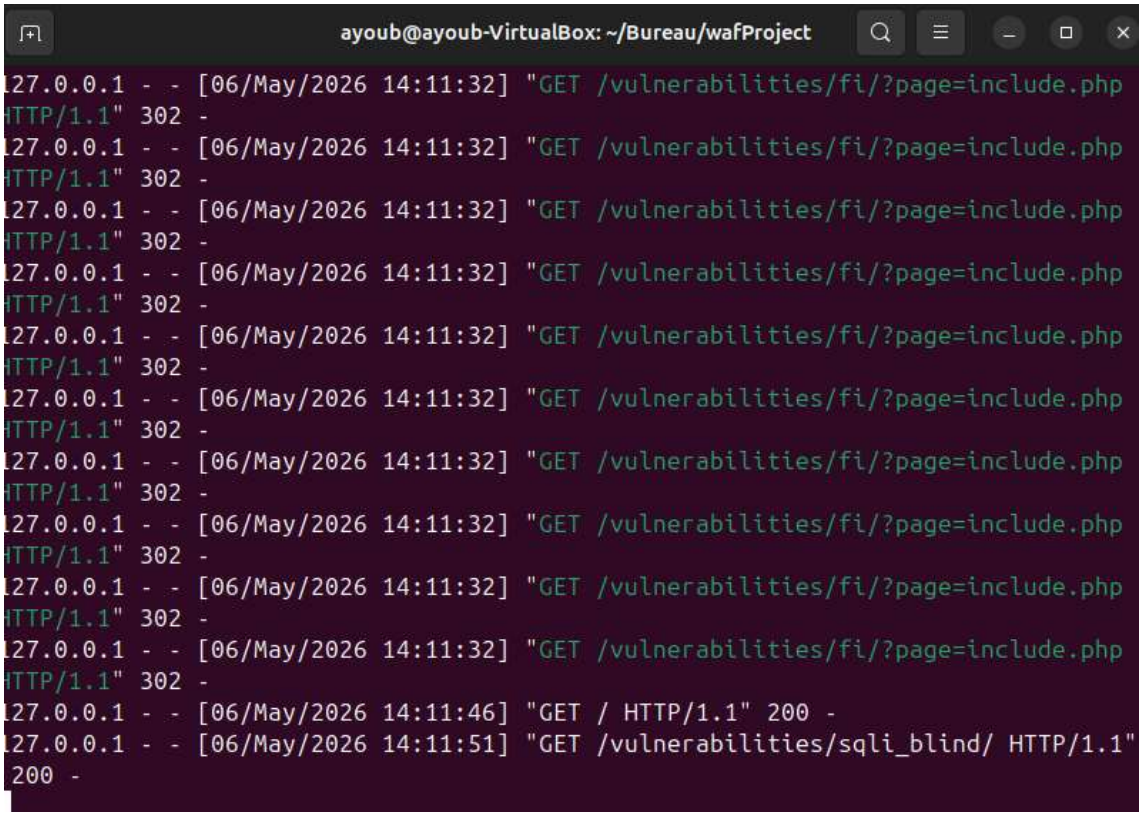
Introduction

Ce chapitre constitue le cœur technique du projet (35% de la note). Il explique comment vous avez transformé des concepts théoriques en un outil fonctionnel capable d'intercepter et d'analyser le trafic en temps réel.

1. Architecture du Logiciel (Python & Flask):

Le WAF a été développé en Python en utilisant le micro-framework Flask. Contrairement à un serveur web classique, il est configuré pour agir en **Reverse Proxy**.

- **Interception :** Le script écoute sur le port 5000 et capture chaque requête HTTP entrante.
- **Forwarding :** Si la requête est jugée "saine", elle est transmise au serveur DVWA (port 80) via la bibliothèque requests.



2. Définition des données

Le moteur d'inspection analyse trois composants critiques de chaque requête :

L'URI : Recherche de motifs suspects dans l'URL (ex: ../ pour le Path Traversal).

Les Headers : Vérification du User-Agent ou des Cookies pour détecter des outils de scan automatiques.

Le Body (POST Data) : Analyse du contenu des formulaires.

Élément technique crucial : L'utilisation de urllib.parse.unquote pour décoder les caractères spéciaux (comme %27 pour ') afin d'éviter que les attaquants ne contournent le WAF via l'encodage URL.

3. Logique de Détection et Signatures (Regex)

La détection repose sur des **Expressions Régulières (Regex).** Voici les motifs implémentés :

- **SQL Injection :** Détection de mots-clés (SELECT, UNION, UPDATE) et de tautologies (OR 1=1).
- **Cross-Site Scripting (XSS) :** Détection de balises <script>, d'événements JavaScript (onerror, onload) et d'alertes.

```
# 1. Détection par mots-clés simples
mots_interdits = [
    "select", "union", "drop", "insert", "script",
    "alert(", "onerror=", "1=1", "or 1=1"
]
for mot in mots_interdits:
    if mot in data:
        print(f"[-] DEBUG: Mot interdit détecté -> {mot}")
        return True

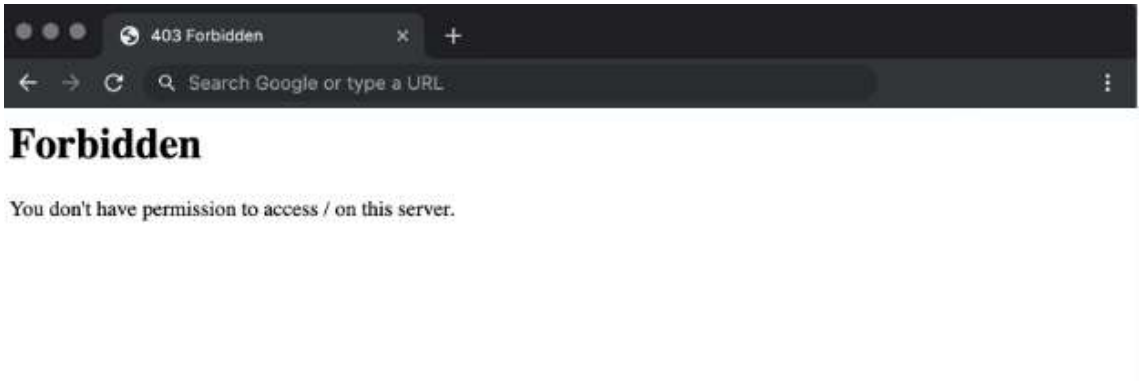
# 2. Détection avancée par Expressions Régulières (Regex)
patterns = [
    r"<\s*script\s*>",                # Balise <script> avec espaces
    r"(\%27)|(\')|(\-\-)|(\%23)|(#)",  # Caractères spéciaux SQL (', --, #)
    r"\b(or|and)\b.*\b\d+=\d+\b",      # Logiques d'injection type OR 1=1 ou AND 5=5
    r"union.*select",                 # Tentatives d'unions complexes
]

for p in patterns:
    if re.search(p, data, re.IGNORECASE):
        print(f"[-] DEBUG: Regex match -> {p}")
        return True

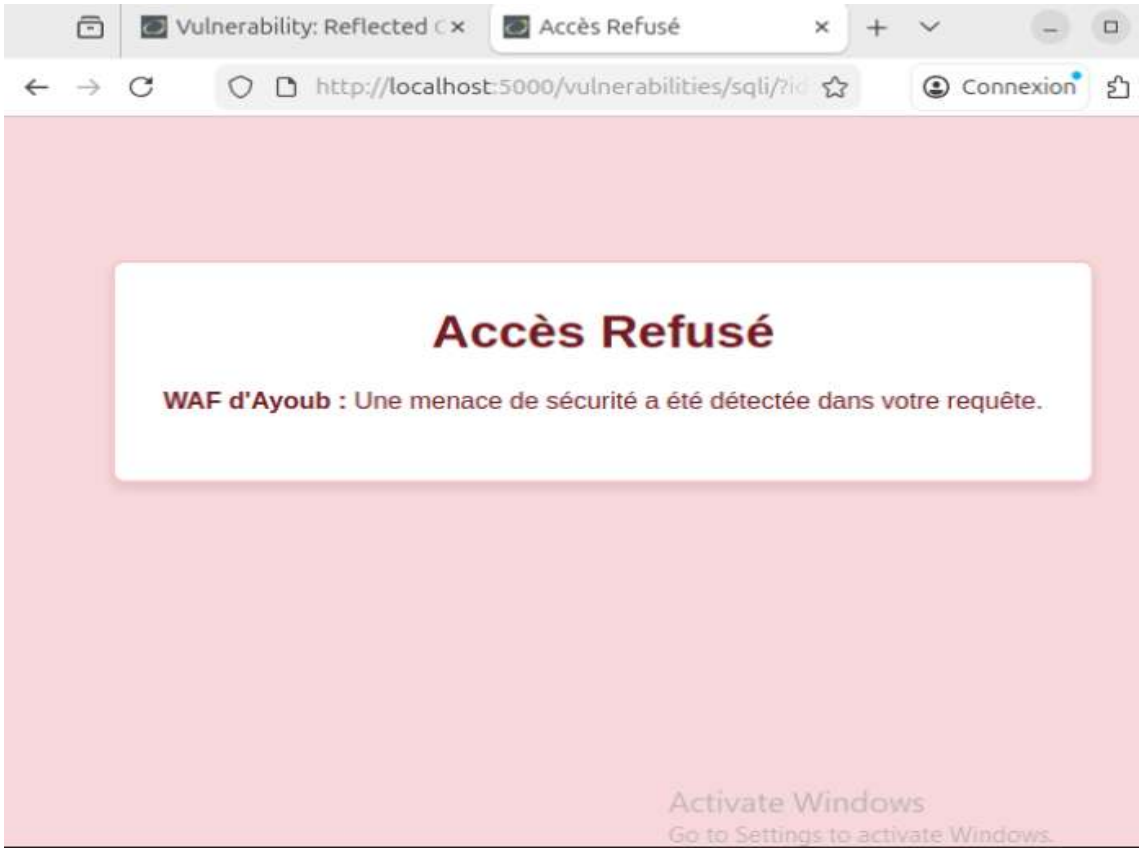
return False
```

4. Gestion du Blocage et Logging

Lorsqu'une menace est détectée :
Réponse HTTP 403 : Le WAF interrompt la connexion et renvoie un code d'erreur "Forbidden".



Page de blocage personnalisée : Une page HTML conçue par vos soins est affichée à l'attaquant.



Journalisation (Logging) : L'attaque est inscrite dans waf.log.

```
ayoub@ayoub-VirtualBox:~/Bureau/wafProject$ ls
my_waf.py  venv  waf.log
ayoub@ayoub-VirtualBox:~/Bureau/wafProject$ cat waf.log
[2026-05-06 23:26:44] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: id=%27+OR+1%3D1&Submit=Submit | BODY: {}
[2026-05-06 23:26:51] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: id=%27+OR+1%3D1&Submit=Submit | BODY: {}
[2026-05-06 23:26:59] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: id=1%27+OR+%271%27%3D%271&Submit=Submit | BO
DY: {}
[2026-05-06 23:27:58] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: name=%3Cscript%3E | BODY: {}
[2026-05-06 23:30:33] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: name=%3Cscript | BODY: {}
[2026-05-06 23:33:53] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fs
cript%3E | BODY: {}
[2026-05-06 23:36:44] IP: 127.0.0.1 | Raison: Signature/Regex Match | Payload: URL: id=or+1%3D1&Submit=Submit | BODY: {}
```

Conclusion

En conclusion, le développement de ce WAF personnalisé démontre qu'avec une logique de programmation rigoureuse et une connaissance précise des signatures d'attaques, il est possible de créer un bouclier efficace et léger. Bien que ce moteur repose sur une détection statique par Regex, sa capacité à inspecter chaque octet des données entrantes et à consigner les tentatives d'intrusion constitue une première ligne de défense robuste avant d'envisager des solutions plus complexes.

CHAPITRE 4 :
PREUVE DE CONCEPT (POC) ET VALIDATION

Introduction

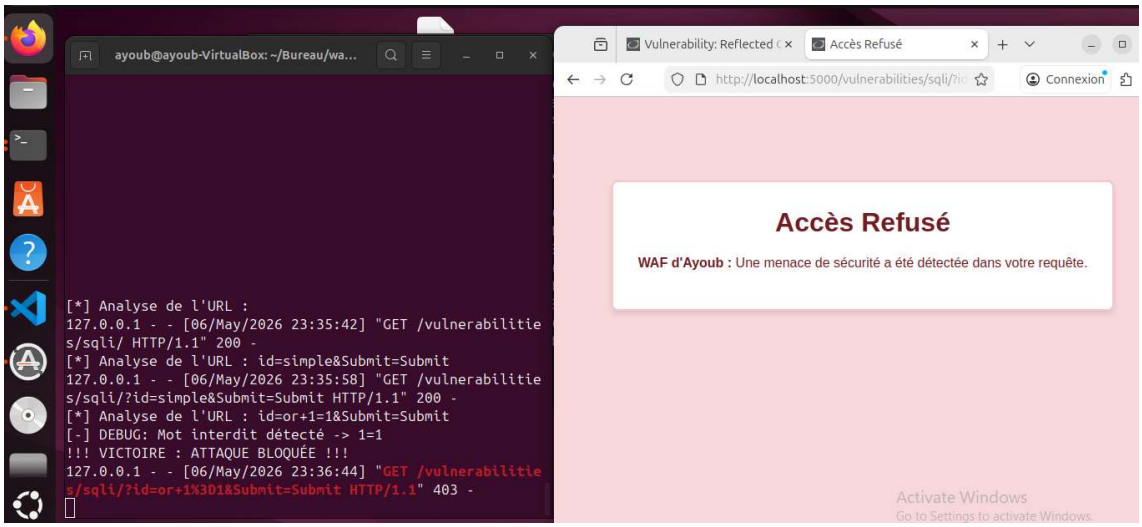
Ce chapitre démontre l'efficacité des deux solutions de défense (Custom et ModSecurity) face à des cyberattaques réelles. Il présente les scénarios de test, les résultats de blocage et l'analyse des logs générés.

1. Scénarios d'attaques manuelles et résultats:

Nous avons confronté notre WAF personnalisé aux mêmes vecteurs d'attaque utilisés lors de la phase 1 (sans protection).

1.1. Blocage de l'Injection SQL (SQLi):

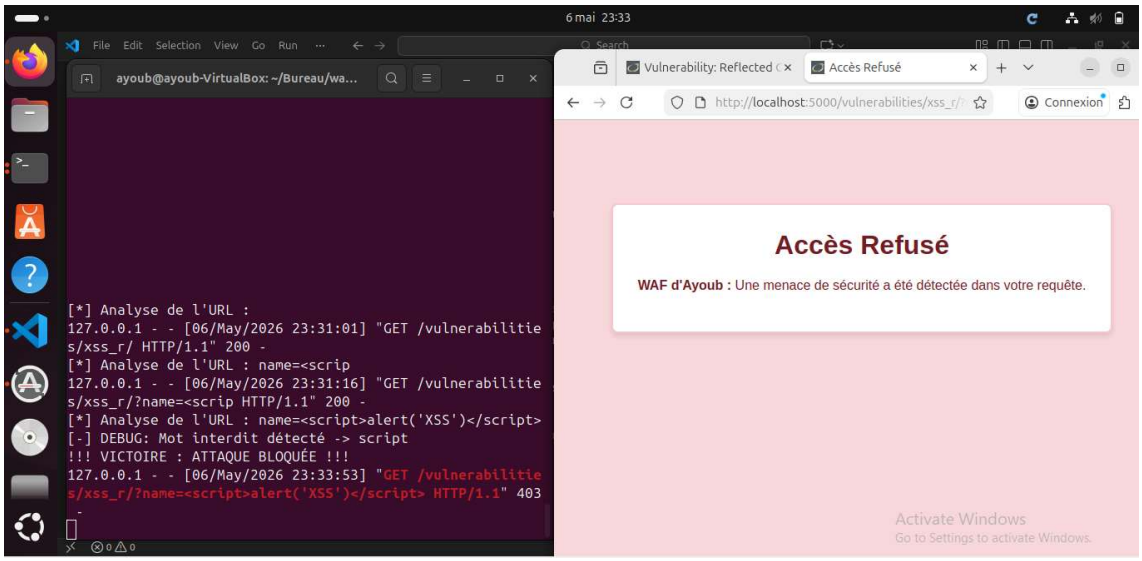
Vecteur utilisé : 1' OR '1'='1' # dans le champ "User ID". ***Comportement du WAF*** : La Regex \b(or|and)\b.*\b\d+=\d+\b a détecté la tautologie. La requête a été immédiatement stoppée avant d'atteindre la base de données de DVWA.



1.2. Blocage du Cross-Site Scripting (XSS):

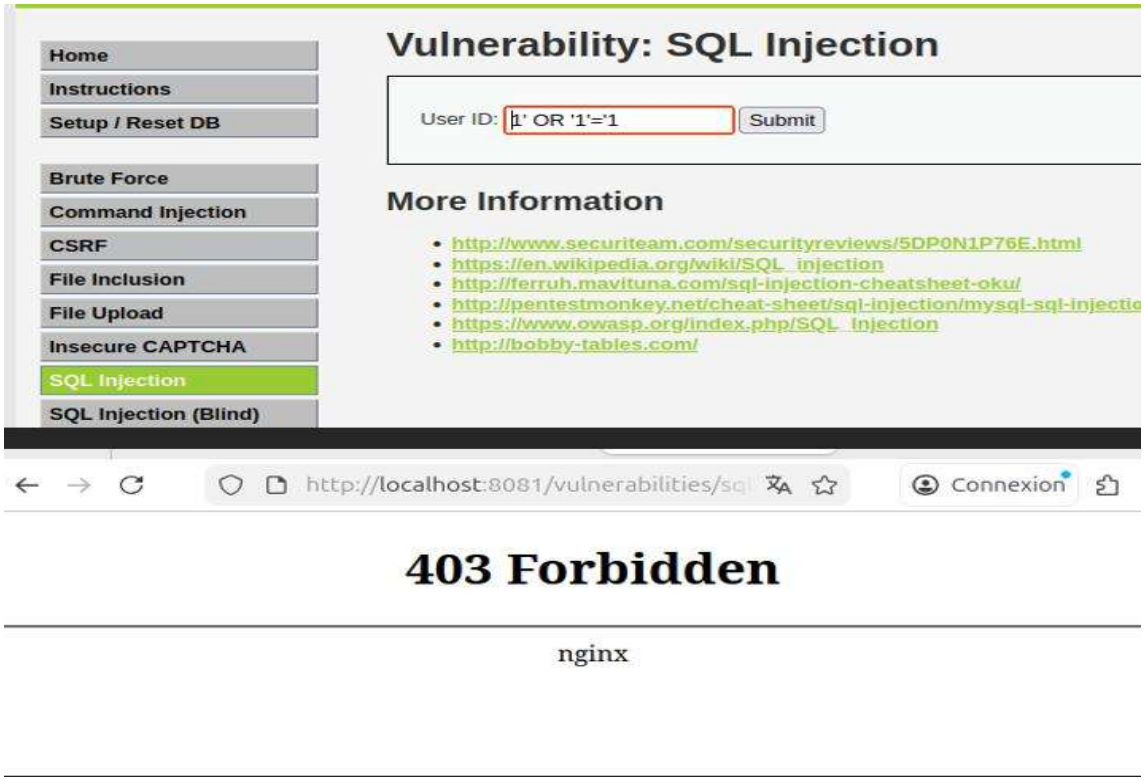
Vecteur utilisé : <script>alert('Ayoub_WAF_Test')</script>

Comportement du WAF : Le moteur d'inspection a identifié la balise <script> dans le corps de la requête POST.



2. Validation du WAF Open-Source ModSecurity:

Exemple de SQLi :

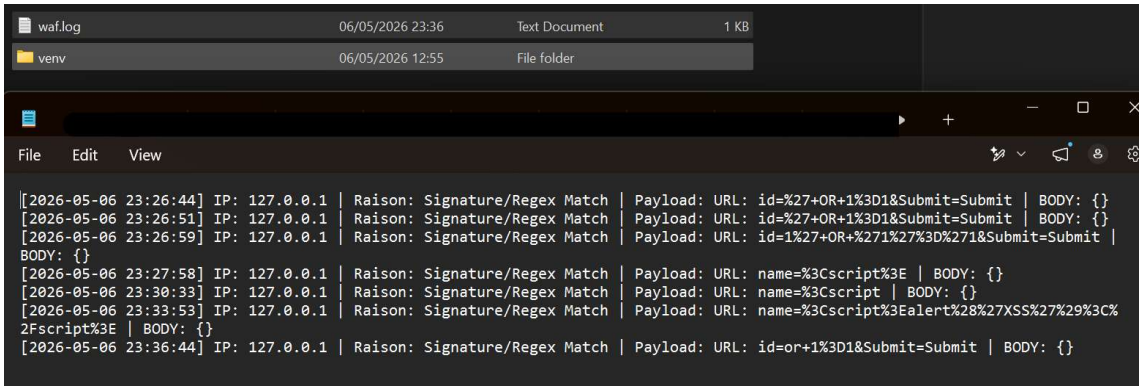


3. Analyse de la Journalisation (Logging & Alerting)

La journalisation est le point de preuve final de l'activité du WAF

3.1. Traçabilité dans le WAF Custom :

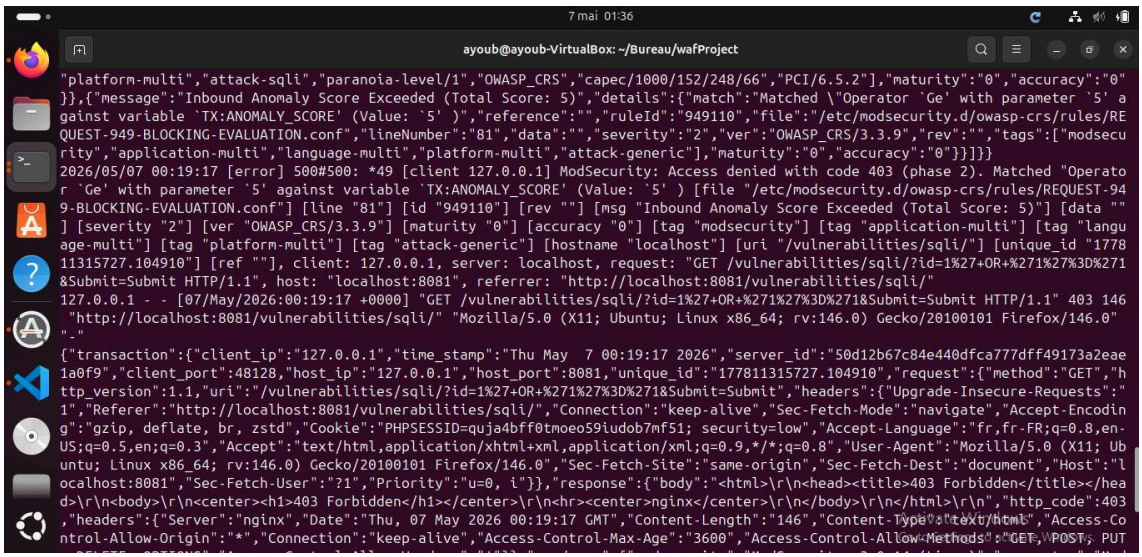
Chaque blocage effectué par notre script Python est consigné pour audit.



Exemple de xss

3.2. Logs de ModSecurity:

- En comparaison, les logs de ModSecurity (error.log ou modsec_audit.log) sont beaucoup plus verbeux et détaillés, incluant les scores de criticité de chaque requête.

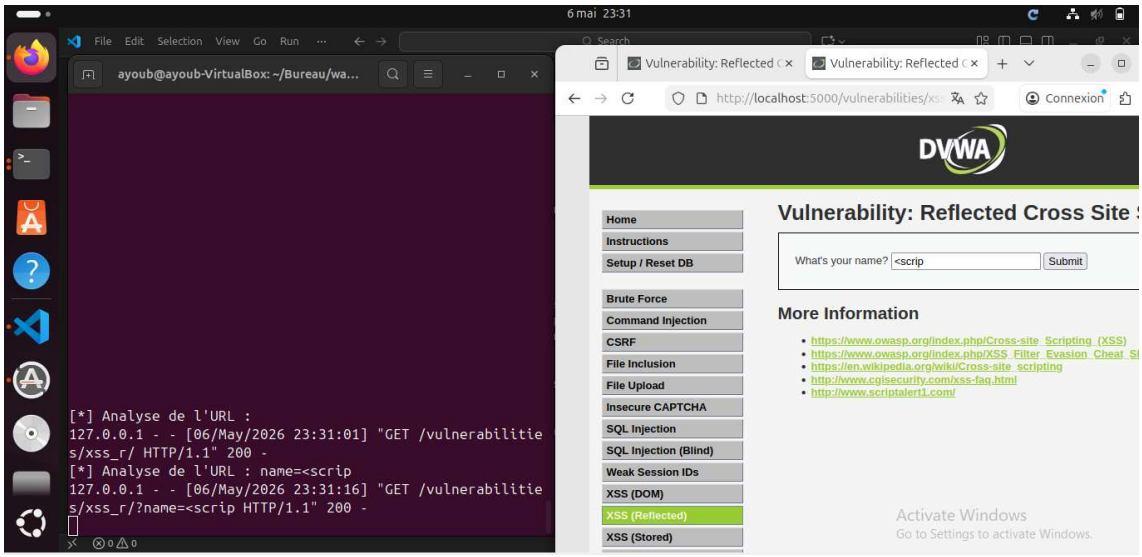


4. Test de légitimité

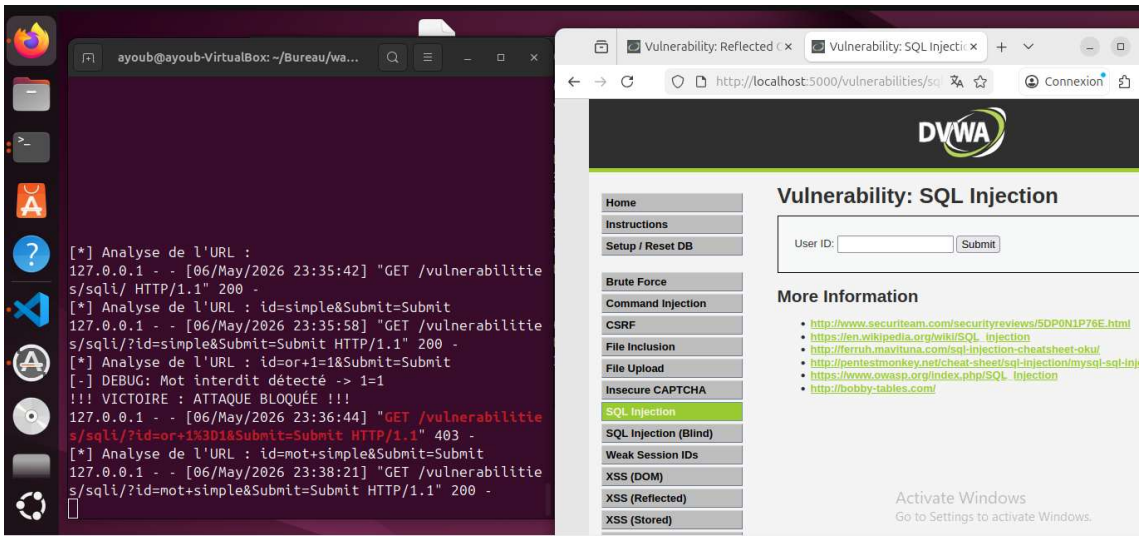
4.1. ModSecurity Edition



4.2. Custum WAF edition



XSS


ayoub@ayoub-VirtualBox: ~/Bureau/wa...
[*] Analyse de l'URL :
127.0.0.1 - - [06/May/2026 23:35:42] "GET /vulnerabilities/sqli/ HTTP/1.1" 200 -
[*] Analyse de l'URL : id=simple&Submit=Submit
127.0.0.1 - - [06/May/2026 23:35:58] "GET /vulnerabilities/sqli/?id=simple&Submit=Submit HTTP/1.1" 200 -
[*] Analyse de l'URL : id=or+1=1&Submit=Submit
[-] DEBUG: Mot interdit détecté -> 1=1
!!! VICTOIRE : ATTAQUE BLOQUÉE !!!
127.0.0.1 - - [06/May/2026 23:36:44] "GET /vulnerabilities/sqli/?id=or+1%3D1&Submit=Submit HTTP/1.1" 403 -
[*] Analyse de l'URL : id=mot+simple&Submit=Submit
127.0.0.1 - - [06/May/2026 23:38:21] "GET /vulnerabilities/sqli/?id=mot+simple&Submit=Submit HTTP/1.1" 200 -
Vulnerability: Reflected
Vulnerability: SQL Injecti
http://localhost:5000/vulnerabilities/sq
Connexion
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
Vulnerability: SQL Injection
User ID:
Submit
More Information
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
https://en.wikipedia.org/wiki/SQL_injection
http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/
http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-inj
https://www.owasp.org/index.php/SQL_Injection
http://bobby-tables.com/
Activate Windows
Go to Settings to activate Windows.

SQLi

Conclusion

Les tests réalisés démontrent que le WAF personnalisé remplit ses fonctions de base avec succès en bloquant les attaques les plus courantes. Toutefois, l'utilisation de ModSecurity met en évidence la supériorité d'une solution maintenue par une communauté, capable de détecter des variantes d'attaques plus subtiles. Cette dualité de résultats prépare le terrain pour l'analyse comparative finale entre solutions artisanales, open-source et professionnelles

CHAPITRE 5 :
L'ULTIME COMPARAISON (ANALYSE ANALYTIQUE)

Introduction

Ce chapitre propose une étude comparative entre les trois piliers de la protection applicative : la solution artisanale (WAF Custom), la solution open-source (ModSecurity) et la solution leader du marché (FortiWeb). L'objectif est d'identifier les cas d'usage optimaux pour chaque technologie

1. Tableau Comparatif de Synthèse :

Le tableau suivant récapitule les performances et caractéristiques observées lors de nos phases de tests :

CRITÈRE	WAF Custom Python · Maison	ModSecurity Nginx · OWASP CRS	FortiWeb Fortinet · Enterprise
Détection	Regex	Signatures CRS	Machine Learning IA
Déploiement	Immédiat	Moyen	Complexe
Faux positifs	✕ Très élevé · Rigide	– Modéré · Tuning requis	✓ Optimal · Auto-apprentissage
Vitesse d'inspection	Faible · Interpréteur	Rapide · C/C++	Ultra · ASIC
Coût de licence	Gratuit	Open-source	Élevé · Opex/Capex
Intégration DevSecOps	Totale · Code-centric	Bonne · Fichiers conf	Excellente · API/Cloud

IDÉAL POUR	IDÉAL POUR	IDÉAL POUR
Prototypage & recherche	**Production open-source**	**Entreprise & cloud critique**
Contrôle total du code, adapté aux environnements de lab (DVWA/PFE).	Standard industriel, couverture OWASP robuste, sans coût de licence.	Performances maximales, IA adaptive, conformité réglementaire (PCI-DSS).

2. Analyse Critique : "Signatures" vs "Machine Learning"

2.1. Les limites de l'approche par signatures (Custom & ModSecurity) :

Nos tests au Chapitre IV ont montré que le WAF Custom et ModSecurity excellent pour bloquer des attaques connues (SQLi/XSS standards). Cependant, ils partagent une faiblesse commune : **la rigidité.**

- Si un attaquant utilise une technique d'obfuscation inédite ou une attaque **Zero-Day**, la signature ne correspondra plus et l'attaque passera (Faux Négatif).

2.2. La supériorité du Machine Learning (FortiWeb) :

Contrairement à nos scripts, **FortiWeb** ne se contente pas de chercher des "mots interdits". Il construit un modèle mathématique du trafic normal :

- Il analyse la longueur des champs, le type de caractères habituels et la fréquence des requêtes.
- **Résultat** : Il est capable de bloquer une anomalie comportementale même si aucune signature de virus n'existe encore.

3. Analyse Critique : "Signatures" vs "Machine Learning"

3.1. La maintenance (Le point faible du WAF Custom)

Le WAF développé en Python nécessite une mise à jour manuelle du code pour chaque nouvelle menace détectée. Dans un environnement de production avec 100 applications, cela devient impossible à gérer.

3.2. Centralisation et Reporting

Alors que ModSecurity et notre WAF génèrent des logs textuels bruts (voir waf.log), FortiWeb offre un dashboard centralisé. Cette visibilité est cruciale pour une équipe SOC afin de corréler les attaques en temps réel sur l'ensemble de l'entreprise.

4. Cas d'usage et Préconisations :

WAF Custom : Idéal pour l'apprentissage, le prototypage rapide ou des besoins de filtrage extrêmement spécifiques et légers.

ModSecurity : La solution de référence pour les PME et les serveurs web isolés cherchant un excellent rapport sécurité/prix.

FortiWeb : Indispensable pour les infrastructures critiques, les banques et l'e-commerce à fort trafic nécessitant une conformité stricte et une protection proactive.

Conclusion

Cette analyse comparative démontre que si le WAF personnalisé constitue un excellent outil pédagogique et une première barrière efficace, il ne peut se substituer à la robustesse d'un WAF industriel. Le passage d'une détection par Regex à une protection par Machine Learning représente le saut technologique nécessaire pour faire face aux menaces persistantes avancées (APT).

Conclusion générale

Ce projet, réalisé dans le cadre du module de sécurité applicative et de l'étude de FortiWeb, nous a permis de comprendre les mécanismes de protection en couche 7 à travers la mise en place et l'analyse de différentes solutions WAF.

L'objectif principal était de sécuriser une plateforme DVWA tout en étudiant les étapes d'interception, d'analyse et de blocage du trafic HTTP.

Le travail s'est basé sur trois axes principaux : la création d'une infrastructure conteneurisée avec Docker, le déploiement de ModSecurity avec les règles OWASP CRS, puis le développement d'un WAF personnalisé en Python fonctionnant comme Reverse Proxy.

Cette dernière partie a constitué l'aspect le plus formateur du projet, notamment grâce à l'utilisation des expressions régulières et à la gestion du filtrage du trafic.

L'analyse comparative avec FortiWeb nous a permis d'identifier les différences entre une détection statique basée sur des signatures et une protection avancée reposant sur le Machine Learning.

Cette expérience montre qu'un WAF développé manuellement représente un excellent outil pédagogique, tandis que les solutions professionnelles restent indispensables pour répondre aux besoins de performance, de maintenance et de sécurité des infrastructures modernes.

Comme perspectives, ce projet pourrait être amélioré par l'ajout d'un module d'analyse comportementale ou par l'intégration de fonctionnalités avancées comme l'inspection SSL.

Ce travail constitue ainsi une base solide pour notre formation et notre future carrière en cybersécurité.

Codes

```
#my_waf.py
from flask import Flask, request, Response
import requests
import re
import datetime
from urllib.parse import unquote

app = Flask(__name__)

TARGET_URL = "http://localhost:3000"

def ecrire_log(ip, payload, raison):
    date = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("waf.log", "a", encoding="utf-8") as f:
        f.write(f"[{date}] IP: {ip} | Raison: {raison} | Payload: {payload}\n")

def est_malveillant(donnees):
    if not donnees:
        return False

    data = unquote(donnees).lower()

    mots_interdits = [
        "select","union","drop","insert",
        "script","alert(","onerror=",
        "1=1","or 1=1"
    ]

    for mot in mots_interdits:
        if mot in data:
            return True

    patterns = [
        r"<\s*script\s*>",
        r"(\%27)|(\')|(\-\-)|(\%23)|(#)",
        r"\b(or|and)\b.*\b\d+=\d+\b",
        r"union.*select"
    ]

    for p in patterns:
```

```
        if re.search(p, data, re.IGNORECASE):
            return True

    return False

@app.route('/', defaults={'path': ''},
methods=["GET","POST","PUT","DELETE"])
@app.route('/<path:path>', methods=["GET","POST","PUT","DELETE"])
def proxy(path):
    request.get_data()

    url_params = request.query_string.decode()
    body_data = request.form.to_dict()
    body_str = str(body_data) + request.get_data().decode()

    if est_malveillant(url_params) or est_malveillant(body_str):

        payload_complet = f"URL: {url_params} | BODY: {body_str}"
        ecrire_log(
            request.remote_addr,
            payload_complet,
            "Signature/Regex Match"
        )

        page_blocage = """
        <html>
            <head>
                <meta charset="utf-8">
                <title>Accès Refusé</title>
            </head>
            <body style="font-family:Arial;text-align:center;margin-
top:100px;">
                <h1>Accès Refusé</h1>
                <p>Requête bloquée par le WAF.</p>
            </body>
        </html>
        """

        return Response(page_blocage, status=403, mimetype='text/html')
```

```
    url = f"{TARGET_URL}/{path}"

    resp = requests.request(
        method=request.method,
        url=url,
        headers={
            key:value for (key,value)
            in request.headers if key != 'Host'
        },
        data=request.get_data(),
        cookies=request.cookies,
        allow_redirects=False
    )

    excluded_headers = [
        'content-encoding',
        'content-length',
        'transfer-encoding',
        'connection'
    ]

    headers = [
        (name,value) for (name,value)
        in resp.raw.headers.items()
        if name.lower() not in excluded_headers
    ]

    return Response(resp.content, resp.status_code, headers)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Bibliographie et webographie

Les photo, Icon et design et autre :

https://www.canva.com/

https://coolors.co/

https://www.freepik.com/

Information des outils de modélisation et autre :

https://www.wikipedia.org

Chatgpt :

https://openai.com/blog/chatgpt

CLAUDE

https://Claude.ai